

Practical:4

Aim

To design and implement ETL (Extract → Transform → Load) and ELT (Extract → Load → Transform) pipelines using the same retail sales dataset and compare their execution performance, resource utilization, data lineage, maintainability, and architectural suitability.

Code & Output

```
In [1]: !pip install -q pandas duckdb psutil
```

```
In [2]: import pandas as pd
import duckdb
import time
import psutil
import os
```

Create Sample Dataset

```
In [10]: data = {
    "Transaction_ID": [1001, 1002, 1003],
    "Store_Code": ["A01", "B04", "A01"],
    "Store_Name": ["Mumbai Store ", " Delhi Store ", " Mumbai Store "],
    "Product": ["Laptop", "Mouse", "Keyboard"],
    "Quantity": ["5", "10", "3"],
    "Unit_Price": ["199.50", "399.25", "100.00"],
    "Sale_Date": ["2024-01-01", "2024-01-02", "2024-01-03"],
    "Customer_ID": ["C001", "C002", "C003"]
}

df = pd.DataFrame(data)

df.to_csv("data/retail_sales.csv", index=False)

df
```

```
Out[10]:
```

	Transaction_ID	Store_Code	Store_Name	Product	Quantity	Unit_Price	Sale_Date
0	1001	A01	Mumbai Store	Laptop	5	199.50	2024-01-01
1	1002	B04	Delhi Store	Mouse	10	399.25	2024-01-02
2	1003	A01	Mumbai Store	Keyboard	3	100.00	2024-01-03

PART A: ETL Pipeline

Start Timer

```
In [11]: start=time.time()
```

Extract

```
In [12]: df=pd.read_csv("data/retail_sales.csv")

df.head()
```

```
Out[12]:
```

	Transaction_ID	Store_Code	Store_Name	Product	Quantity	Unit_Price	Sale_Date
0	1001	A01	Mumbai Store	Laptop	5	199.50	2024-01-01
1	1002	B04	Delhi Store	Mouse	10	399.25	2024-01-02
2	1003	A01	Mumbai Store	Keyboard	3	100.00	2024-01-03

Transform

```
In [13]: df["Store_Name"]=df["Store_Name"].str.strip()

df["Quantity"]=df["Quantity"].astype(int)

df["Unit_Price"]=df["Unit_Price"].astype(float)

df["Store_Code"]=df["Store_Code"].replace({ "A0
1": "Ahmedabad",
"B04": "Delhi"
})

df["Total"]=df["Quantity"]*df["Unit_Price"]

df=df.drop_duplicates()

df
```

```
Out[13]:
```

	Transaction_ID	Store_Code	Store_Name	Product	Quantity	Unit_Price	Sale_Date
0	1001	Ahmedabad	Mumbai Store	Laptop	5	199.50	2024-01-01
1	1002	Delhi	Delhi Store	Mouse	10	399.25	2024-01-02
2	1003	Ahmedabad	Mumbai Store	Keyboard	3	100.00	2024-01-03

Load into DuckDB

```
In [14]: con=duckdb.connect("warehouse.duckdb")

con.execute("DROP TABLE IF EXISTS sales_etl")

con.execute("CREATE TABLE sales_etl AS SELECT * FROM df")
```

Out[14]: <_duckdb.DuckDBPyConnection at 0x24b81348bf0>

End Timer

```
In [15]: etl_time=time.time()-start

print("ETL Time:",etl_time)
```

ETL Time: 43.97659611701965

Table

```
In [16]: con.execute("SELECT * FROM sales_etl").fetchdf()
```

Out[16]:

	Transaction_ID	Store_Code	Store_Name	Product	Quantity	Unit_Price	Sale_Date
0	1001	Ahmedabad	Mumbai Store	Laptop	5	199.50	2024-01-01
1	1002	Delhi	Delhi Store	Mouse	10	399.25	2024-01-02
2	1003	Ahmedabad	Mumbai Store	Keyboard	3	100.00	2024-01-03

PART B : ELT Pipeline

Start Timer

```
In [17]: start=time.time()
```

Load Raw CSV

```
In [18]: con.execute("DROP TABLE IF EXISTS staging")

con.execute("""

CREATE TABLE staging AS

SELECT *

FROM read_csv_auto('data/retail_sales.csv')

""")
```

Out[18]: <_duckdb.DuckDBPyConnection at 0x24b81348bf0>

SQL Transformation

```
In [19]: con.execute("DROP TABLE IF EXISTS sales_elt")
```

Out[19]: <_duckdb.DuckDBPyConnection at 0x24b81348bf0>

```
In [20]: con.execute("""
CREATE TABLE sales_elt AS
SELECT
Transaction_ID,
TRIM(Store_Name) AS Store_Name,
CASE
WHEN Store_Code='A01' THEN 'Ahmedabad'
WHEN Store_Code='B04' THEN 'Delhi'
END AS Store,
CAST(Quantity AS INTEGER) AS Quantity,
CAST(Unit_Price AS DOUBLE) AS Price,
CAST(Quantity AS INTEGER)
*
CAST(Unit_Price AS DOUBLE) AS Total,
Sale_Date,
Customer_ID
FROM staging
""")
```

Out[20]: <_duckdb.DuckDBPyConnection at 0x24b81348bf0>

End Timer

```
In [21]: elt_time=time.time()-start
print("ELT Time:",elt_time)
```

ELT Time: 86.12979698181152

Table

```
In [22]: con.execute("SELECT * FROM sales_elt").fetchdf()
```

Out[22]:

	Transaction_ID	Store_Name	Store	Quantity	Price	Total	Sale_Date	Custo
0	1001	Mumbai Store	Ahmedabad	5	199.50	997.5	2024-01-01	
1	1002	D lhi Store	Delhi	10	399.25	3992.5	2024-01-02	
2	1003	Mumbai Store	Ahmedabad	3	100.00	300.0	2024-01-03	

C  C

Performance Measurement

CPU Usage

```
In [23]: cpu=psutil.cpu_percent(interval=1)

print("CPU Usage:",cpu,"%")
```

CPU Usage: 29.3 %

RAM Usage

```
In [24]: ram=psutil.virtual_memory()

print("RAM Used:",ram.percent,"%")
```

RAM Used: 87.1 %

Database Size

```
In [25]: size=os.path.getsize("warehouse.duckdb")/(1024)

print("Database Size:",round(size,2),"KB")
```

Database Size: 12.0 KB

Compare Execution Time

```
In [26]: print("ETL :",round(etl_time,4),"seconds")

print("ELT :",round(elt_time,4),"seconds")
```

ETL : 43.9766 seconds

ELT : 86.1298 seconds

Tables

ETL

```
In [27]: con.execute("SELECT * FROM sales_etl").fetchdf()
```

Out[27]:

	Transaction_ID	Store_Code	Store_Name	Product	Quantity	Unit_Price	Sale_Date
0	1001	Ahmedabad	Mumbai Store	Laptop	5	199.50	2024-01-01
1	1002	Delhi	Delhi Store	Mouse	10	399.25	2024-01-02
2	1003	Ahmedabad	Mumbai Store	Keyboard	3	100.00	2024-01-03

ELT

In [28]:

```
con.execute("SELECT * FROM sales_elt").fetchdf()
```

Out[28]:

	Transaction_ID	Store_Name	Store	Quantity	Price	Total	Sale_Date	Custo
0	1001	Mumbai Store	Ahmedabad	5	199.50	997.5	2024-01-01	
1	1002	Delhi Store	Delhi	10	399.25	3992.5	2024-01-02	
2	1003	Mumbai Store	Ahmedabad	3	100.00	300.0	2024-01-03	

Conclusion

ETL performs transformations in Python before loading the data into DuckDB. ELT first loads raw data into DuckDB and performs transformations using SQL. ELT is generally faster and more scalable for analytical workloads.

Learning Outcome

- Understood ETL and ELT architectures.
- Implemented ETL using Pandas.
- Implemented ELT using DuckDB SQL.
- Compared execution performance.
- Measured CPU, RAM and database size.
- Learned the advantages of ELT for modern analytics systems.